Scenario 5: High Application Latency Using Datadog

Objective

Use Datadog APM to identify a slow workload before investigating Kubernetes and applying a focused recovery.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/inject-latency.sh ecommerce 3000
```

The request reaches one backend pod because chaos state is stored per process. Generate application traffic and wait for APM ingestion before troubleshooting. Repeat through the documented port-forward method only when you want every backend pod affected.

Situation

Users report that ecommerce pages load slowly. Do not restart pods before scoping the problem.

Symptoms

- Ecommerce responses are intermittently slow.
- Pods remain Ready.

What You Know

- Datadog reports elevated p95 latency.
- The issue affects `ecommerce-backend` while pods remain Ready.
- The start time does not match an AWS or database change.

Start With Datadog

1. Go to **Dashboards > Dashboard List**, open **ecommerce**, and set the time range to **Past 30 Minutes**. In **p95 Latency**, look for `p95:trace.express.request{env:lab,service:ecommerce-backend}` rising above the normal baseline.
2. Go to **Dashboards > Dashboard List**, open **SRE Lab Scenario Signals**, and compare **Backend p95 Latency** with memory, restarts, and available replicas. The scenario should raise latency without

requiring resource saturation or unhealthy pods.

3. Go to **Monitors > Manage Monitors**, search `[SRE Lab] High p95 latency`, open it, and expand the `ecommerce-backend` group. Record the alert start time and value.

The key scope is slow APM traces for one service while Kubernetes health and resource signals remain near baseline.

Troubleshooting

1. Confirm Kubernetes health.

```
kubectl get deployment,pods -n ecommerce
```

Expected output: the Deployment is fully available, pods are Ready, and restart counts remain stable. The application is slow but not unavailable.

2. Check resource usage.

```
kubectl top pods -n ecommerce
```

Expected output: CPU and memory remain near their normal baseline. This makes resource saturation and OOM restarts unlikely.

3. Inspect the backend logs during the slow-trace timestamps.

```
kubectl logs deployment/ecommerce-backend -n ecommerce --since=15m
```

Expected output: requests complete without crashes, but response timing corresponds to the slow traces. There should be no probe failure, OOM, or restart pattern.

4. Check the application's current chaos state.

```
curl -s "https://ecommerce.$(cat .lab-domain)/api/chaos"
```

Expected output: JSON containing `"latencyMs":3000`. This in-memory setting explains traces taking approximately three seconds while Kubernetes remains healthy.

Important Clues

Determine why one or more backend containers add time before every response. Use trace duration and the chaos-state endpoint as evidence.

```
curl -s https://ecommerce.$(cat .lab-domain)/api/chaos
```

Root Cause

Record the workload behavior that accounts for the trace duration while Kubernetes resource metrics remain normal. Confirm it with the instructor.

Fix

Clear the workload's injected latency. Chaos state is per process, so repeat until every backend pod is reset, or reset each pod through port-forwarding as documented in the README.

```
./scripts/chaos/reset.sh ecommerce
```

Validation

Generate normal requests. Confirm p95 latency and traces return to baseline, pods remain healthy, Kubernetes metrics remain normal, and the Datadog monitor recovers.

DevOps Lesson

Use traces to isolate the slow workload before taking action. A restart may hide evidence without explaining the cause.

STAR Method

Situation

Summarize the latency report and initial scope.

Task

Explain the need to identify the affected workload before acting.

Action

Describe the dashboard, trace, Kubernetes, log, and workload-state evidence.

Result

State how normal latency and healthy pods were verified.

Natural Spoken Version

Use the matching example in DevOps STAR Scenarios after completing the lab.

Brief STAR Example

Users reported slow responses from one of our applications while its EKS pods remained healthy. My task was to isolate the source before restarting anything. Datadog APM showed requests taking approximately three seconds, while CPU, memory, readiness, and restarts stayed normal. I found and cleared a 3000 ms application delay setting, after which p95 latency returned to baseline and the monitor recovered.